CREATE INDEX

Purpose

Use the CREATE INDEX statement to create an index on:

- One or more columns of a table, a partitioned table, an index-organized table, or a cluster
- One or more scalar typed object attributes of a table or a cluster
- A nested table storage table for indexing a nested table column

An index is a schema object that contains an entry for each value that appears in the indexed column(s) of the table or cluster and provides direct, fast access to rows. The maximum size of a single index entry is dependent on the block size of the database.

Prerequisites

To create an index in your own schema, one of the following conditions must be true:

- The table or cluster to be indexed must be in your own schema.
- You must have the INDEX object privilege on the table to be indexed.
- You must have the CREATE ANY INDEX system privilege.
- The CREATE INDEX statement is used to create indexes in tables.
- Indexes are used to retrieve data from the database more quickly than otherwise. The users cannot see the indexes, they are just used to speed up searches/queries.
- **Note:** Updating a table with indexes takes more time than updating a table without (because the indexes also need an update). So, only create indexes on columns that will be frequently searched against.

CREATE INDEX Syntax

Creates an index on a table. Duplicate values are allowed:

CREATE INDEX *index_name*
ON *table_name* (*column1*, *column2*, ...);

CREATE UNIQUE INDEX Syntax

Creates a unique index on a table. Duplicate values are not allowed:

CREATE UNIQUE INDEX *index_name*
ON *table_name* (*column1*, *column2*, ...);

CREATE INDEX Example

The SQL statement below creates an index named "idx_lastname" on the "LastName" column in the "Persons" table:

```
CREATE INDEX idx_lastname
ON Persons (LastName);
```

If you want to create an index on a combination of columns, you can list the column names within the parentheses, separated by commas:

```
CREATE INDEX idx_pname
ON Persons (LastName, FirstName);
```

OUTPUT:

Index created

DROP INDEX Statement

The DROP INDEX statement is used to delete an index in a table.

```
DROP INDEX index_name;
```

Syntax for Oracle

In Oracle the code is a little bit more tricky.

You will have to create an auto-increment field with the sequence object (this object generates a number sequence).

Use the following CREATE SEQUENCE syntax:

```
CREATE SEQUENCE seq_person
MINVALUE 1
START WITH 1
INCREMENT BY 1
CACHE 10;
```

The code above creates a sequence object called seq_person, that starts with 1 and will increment by 1. It will also cache up to 10 values for performance. The cache option specifies how many sequence values will be stored in memory for faster access.

To insert a new record into the "Persons" table, we will have to use the nextval function (this function retrieves the next value from seq_person sequence):

```
INSERT INTO Persons (Personid,FirstName,LastName)
VALUES (seq_person.nextval,'Lars','Monsen');
```

PL/SQL searched CASE statement

PL/SQL provides a special CASE statement called *searched CASE statement*. The syntax of the PL/SQL searched CASE statement is as follows:

```
[<<label_name>>]
CASE
   WHEN search_condition_1 THEN sequence_of_statements_1;
   WHEN search_condition_2 THEN sequence_of_statements_2;
   ...
   WHEN search_condition_N THEN sequence_of_statements_N;
  [ELSE sequence_of_statements_N+1;]
END CASE [label_name];
```

The searched CASE statement has no selector. Each WHEN clause in the searched CASE statement contains a search condition that returns a Boolean value.

The search condition is evaluated sequentially from top to bottom. If a search condition evaluates to TRUE, the sequence of statements in the corresponding WHEN clause is executed and the control is passed to the next statement, therefore, the subsequent search conditions are ignored.

If no search condition evaluates to TRUE, the sequence of statements in the ELSE clause will be executed.

The following example demonstrates the PL/SQL CASE statement. We'll use the employees table

```
DECLARE
  n_salary EMPLOYEES.SALARY%TYPE;
  n_emp_id EMPLOYEES.EMPLOYEE_ID%TYPE := 200;
  v_msg    VARCHAR(20);
BEGIN
  SELECT salary
  INTO n_salary
  FROM employees
  WHERE employee_id = n_emp_id;

  CASE
    WHEN n_salary < 2000 THEN
      v_msg := 'Low';
    WHEN n_salary >= 2000 and n_salary <=3000 THEN
      v_msg := 'Fair';
    WHEN n_salary >= 3000 THEN
      v_msg := 'High';
  END CASE;
  DBMS_OUTPUT.PUT_LINE(v_msg);
END;
/
```

PL/SQL LOOPS

PL/SQL LOOP statement is an iterative control statement that allows you to execute a sequence of statements repeatedly like WHILE and FOR loop.

PL/SQL provides the following types of loop to handle the looping requirements. Click the following links to check their detail.

S.No	Loop Type & Description
1	PL/SQL Basic LOOP In this loop structure, sequence of statements is enclosed between the LOOP and the END LOOP statements. At each iteration, the sequence of statements is executed and then control resumes at the top of the loop.
2	PL/SQL WHILE LOOP Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
3	PL/SQL FOR LOOP Execute a sequence of statements multiple times and abbreviates the code that manages the loop variable.
4	Nested loops in PL/SQL You can use one or more loop inside any another basic loop, while, or for loop.

The simplest form of the LOOP statement consists of the LOOP keyword, a sequence of statements and the END LOOP keywords as shown below:

```
LOOP
  sequence_of_statements;
END LOOP;
```

The following illustrates the PL/SQL LOOP statement with EXIT and EXIT-WHEN statements:

```
LOOP
  ...
  EXIT;
END LOOP;Code language: SQL (Structured Query Language) (sql)
LOOP
  ...
  EXIT WHEN condition;
END LOOP;
```

The following is example of using PL/SQL LOOP statement with EXIT:

Exmp1:

```
DECLARE n_counter NUMBER := 0;
BEGIN
  LOOP
    n_counter := n_counter + 1;
    DBMS_OUTPUT.PUT_LINE(n_counter);
    IF n_counter = 5 THEN
      EXIT;
    END IF;
  END LOOP;
END;
/
```

Exmp2:

```
DECLARE
  n_i NUMBER := 0;
  n_j NUMBER := 0;
BEGIN
  << outer_loop >>
  LOOP
    n_i := n_i + 1;
    EXIT WHEN n_i = 2;
    << inner_loop >>
    LOOP
      n_j := n_j + 1;
      EXIT WHEN n_j = 5;
      DBMS_OUTPUT.PUT_LINE('Outer loop counter ' || n_i);
      DBMS_OUTPUT.PUT_LINE('Inner loop counter ' || n_j);
    END LOOP inner_loop;
  END LOOP outer_loop;
END;
/
```

Note that there must be at least one executable statement between LOOP and END LOOP keywords. The sequence of statements is executed repeatedly until it reaches a loop exit. PL/SQL provides EXIT and EXIT-WHEN statements to allow you to terminate a loop.

- The EXIT forces the loop halts execution unconditionally and passes control to the next statement after the LOOP statement. You typically use the EXIT statement with the IF statement.
- The EXIT-WHEN statement allows the loop to terminate conditionally. When the EXIT-WHEN statement is reached, the condition in the WHEN clause is checked. If the condition evaluates to TRUE, the loop is terminated and control is passed to the next

statement after the keyword END LOOP. If the condition evaluates to FALSE, the loop will continue repeatedly until the condition evaluates to TRUE.

The following program illustrates the concept −

```
DECLARE
  i number(1);
  j number(1);
BEGIN
  << outer_loop >>
  FOR i IN 1..3 LOOP
    << inner_loop >>
    FOR j IN 1..3 LOOP
      dbms_output.put_line('i is: '|| i || ' and j is: ' || j);
    END loop inner_loop;
  END loop outer_loop;
END;
/
```

The following illustrates the PL/SQL WHILE LOOP syntax:

```
WHILE condition
LOOP
  sequence_of_statements;
END LOOP;
```

This example calculates factorial of 10 by using PL/SQL WHILE LOOP statement:

```
DECLARE
 n_counter   NUMBER := 10;
 n_factorial NUMBER := 1;
 n_temp      NUMBER;
BEGIN
 n_temp := n_counter;

 WHILE n_counter > 0
 LOOP
  n_factorial := n_factorial * n_counter;
  n_counter   := n_counter - 1;
 END LOOP;

 DBMS_OUTPUT.PUT_LINE('factorial of ' || n_temp ||
                 ' is ' || n_factorial);
```

END;
/

Output:
factorial of 10 is 3628800

PL/SQL procedure successfully completed.

Introducing to PL/SQL function

PL/SQL function is a named block that returns a value. A PL/SQL function is also known as a subroutine or a subprogram. To create a PL/SQL function, you use the following syntax:

```
CREATE [OR REPLACE] FUNCTION function_name [(
   parameter_1 [IN] [OUT] data_type,
   parameter_2 [IN] [OUT] data_type,
   parameter_N [IN] [OUT] data_type]
    RETURN return_data_type IS
--the declaration statements
BEGIN
   -- the executable statements
   return return_data_type;
   EXCEPTION
    -- the exception-handling statements
END;
/Code language: SQL (Structured Query Language) (sql)
```

Let's examine the syntax of creating a function in greater detail:

You specify the function name function_nameafter the FUNCTIONkeyword. By convention, the function name should start with a verb, for example convert_to_number.

A function may have zero or more than one parameter. You specify the parameter names in the parameter_1, parameter_2, etc. You must specify the data type of each parameter explicitly in the data_type. Each parameter has one of three modes: IN, OUT and IN OUT.

- An IN parameter is a read-only parameter. If the function tries to change the value of the INparameters, the compiler will issue an error message. You can pass a constant, literal, initialized variable, or expression to the function as the INparameter.
- An OUTparameter is a write-only parameter. The OUTparameters are used to return values back to the calling program. An OUTparameter is initialized to a default value of its type when the function begins regardless of its original value before being passed to the function.
- An IN OUT parameter is read and write parameter. It means the function reads the value from an IN OUT parameter, change its value and return it back to the calling program.

The function must have at least one RETURNstatement in the execution section. The RETURN clause in the function header specifies the data type of returned value.

Examples of PL/SQL Function

We are going to create a function named `try_parse` that parses a string and returns a number if the input string is a number or `NULL`if it cannot be converted to a number.

```
CREATE OR REPLACE FUNCTION try_parse(
   iv_number IN VARCHAR2)
 RETURN NUMBER IS
BEGIN
  RETURN to_number(iv_number);
  EXCEPTION
    WHEN others THEN
      RETURN NULL;
END;
```

Code language: SQL (Structured Query Language) (sql)

The `iv_number` is an `IN`parameter whose data type is `VARCHAR2` so that you can pass any string to the `try_parse()` function.

Inside the function, we used the built-in PL/SQL function named `to_number()` to convert a string into a number. If any exception occurs, the function returns NULL in the exception section, otherwise, it returns a number.

Calling PL/SQL Function

The PL/SQL function returns a value so you can use it on the right-hand side of an assignment or in a SELECT statement.

Let’s create an anonymous block to use the try_parse() function.

SET SERVEROUTPUT ON SIZE 1000000;

```
DECLARE
  n_x number;
  n_y number;
   n_z number;
BEGIN
   n_x := try_parse('574');
   n_y := try_parse('12.21');
   n_z := try_parse('abcd');

   DBMS_OUTPUT.PUT_LINE(n_x);
   DBMS_OUTPUT.PUT_LINE(n_y);
   DBMS_OUTPUT.PUT_LINE(n_z);
END;
/
```

We can also use the try_parse() function in a SELECT statement as follows:

SELECT try_parse('1234') FROM dual;

SELECT try_parse('Abc') FROM dual;

Introduction to PL/SQL Procedure

Like a PL/SQL function, a PL/SQL procedure is a named block that does a specific task. PL/SQL procedure allows you to encapsulate complex business logic and reuse it in both database layer and application layer.

The following illustrates the PL/SQL procedure's syntax:

```
PROCEDURE [schema.]name[( parameter[,
parameter...] ) ]
[AUTHID DEFINER | CURRENT_USER]
IS
[--declarations  statements]
BEGIN
--executable statements
[ EXCEPTION
---exception handlers]
END [name];
```

PL/SQL Procedure Header

The section before **`IS`** keyword is called procedure header or procedure signature. The elements in the procedure's header are described as follows:

- **`schema`**: the optional name of the schema that the procedure belongs to. The default is the current user. If you specify a different user, the current user must have privileges to create a procedure in that schema.
- **`name`:** The name of the procedure. The name of the procedure, by convention, should start with a verb e.g., `put_line`
- **`parameters`**: the optional list of parameters. Please refer to the PL/SQL function for more information on parameters with different modes `IN`, `OUT`and `IN OUT`.
- **`AUTHID`:** The optional `AUHTID` determines whether the procedure will execute with the privileges of the owner (`DEFINER`) of the procedure or with the privileges of the current user specified by `CURRENT_USER`.

PL/SQL Procedure Body

Everything after the **`IS`** keyword is known as procedure body. The procedure body has similar syntax with an anonymous block which consists of the declaration, execution and exception sections.

The declaration and exception sections are optional. You must have at least one executable statement in the execution section. The execution section is the where you put the code to implement a given business logic to perform a specific task.

In PL/SQL procedure you can have a `RETURN` statement. However, unlike the `RETURN` statement in the function that returns a value to calling program, the `RETURN` statement in the procedure is used only to halt the execution of the procedure and return control to the caller. The `RETURN` statement in procedure does not take any expression or constant.

Example of PL/SQL Procedure

We're going to develop a procedure named `adjust_salary()` in HR sample database provided by Oracle. We'll update the salary information of employees in the `employees` table by using SQL UPDATE statement.

The following is the source code of the `adjust_salary()` procedure :

How it works.

- The procedure has two parameters: `IN_EMPLOYEE_ID` and `IN_PERCENT`.
- The procedure adjusts the salary of a particular employee specified the `IN_EMPLOYEE_ID` by a given percentage `IN_PERCENT`.
- In the procedure body, we use the `UPDATE` statement to update the salary information.

Let's take a look at how to call this procedure in various context.

```
CREATE OR REPLACE PROCEDURE adjust_salary(
   in_employee_id IN EMPLOYEES.EMPLOYEE_ID%TYPE,
   in_percent IN NUMBER
) IS
BEGIN
   -- update employee's salary
   UPDATE employees
   SET salary = salary + salary * in_percent / 100
   WHERE employee_id = in_employee_id;
END;
```

Calling PL/SQL Procedure

A procedure can call other procedures. A procedure without parameters can be called directly by using EXEC statement or EXECUTE statement followed by the name of the procedure as follows:

EXEC procedure_name();
EXEC procedure_name;

A procedure with parameters can be called by using `EXEC`or `EXECUTE`statement followed by procedure's name and its parameters in the order corresponding to the parameters list of the procedure as shown below:

```
EXEC procedure_name(param1,param2…paramN);
```
Code language: SQL (Structured Query Language) (sql)

Now, we can call `adjust_salary()`procedure as the following statements:

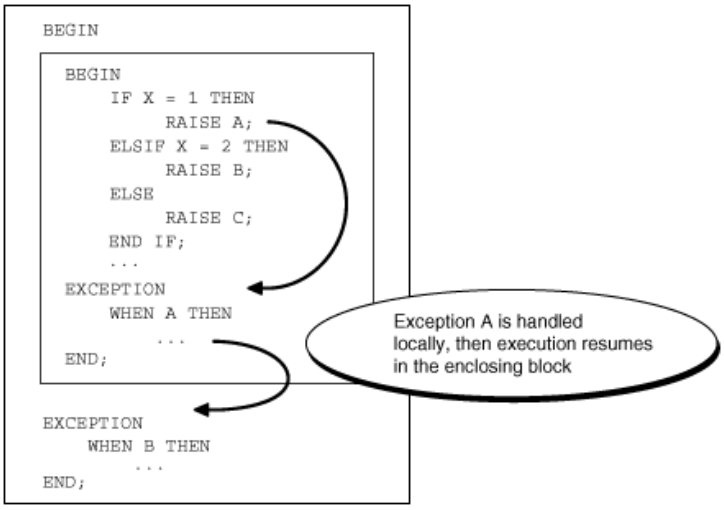
```
SELECT salary FROM employees WHERE employee_id = 200;
-- call procedure
exec adjust_salary(200,5);
-- after adjustment
SELECT salary FROM employees WHERE employee_id = 200;
```

Introducing to PL/SQL Exception

In PL/SQL, any kind of errors is treated as exceptions. An exception is defined as a special condition that changes the program execution flow. The PL/SQL provides you with a flexible and powerful way to handle such exceptions.

PL/SQL catches and handles exceptions by using exception handler architecture. Whenever an exception occurs, it is raised. The current PL/SQL block execution halts, control is passed to a separate section called *exception section*.

In the exception section, you can check what kind of exception has been occurred and handle it appropriately. This exception handler architecture enables separating the business logic and exception handling code hence make the program easier to read and maintain.



There are two types of exceptions:

- **System exception**: the system exception is raised by PL/SQL run-time when it detects an error. For example, `NO_DATA_FOUND`exception is raised if you select a non-existing record from the database.
- **Programmer-defined exception**: the programmer-defined exception is defined by you in a specific application. You can map exception names with specific Oracle errors using the `EXCEPTION_INIT`pragma. You can also assign a number and description to the exception using `RAISE_APPLICATION_ERROR`.

Defining PL/SQL Exception

An exception must be defined before it can be raised. Oracle provides many predefined exceptions in the `STANDARD`package. To define an exception you use `EXCEPTION`keyword as below:

```
EXCEPTION_NAME EXCEPTION;
```
Code language: SQL (Structured Query Language) (sql)

To raise an exception that you've defined you use the `RAISE`statement as follows:

```
RAISE EXCEPTION_NAME;
```

In the exception handler section, you use can handle the exception as usual. The following example illustrates the programmer-defined exceptions

EMP Table:

EMPNO	SAL	DEPTNO
101	5000	10
102	6000	20
103	2000	10
104	4000	20
105	2000	30

By using the above table data ,implement the given plsql block to define exception handling technique.

```
DECLARE
  -- define exceptions
  BELOW_SALARY_RANGE EXCEPTION;
  ABOVE_SALARY_RANGE EXCEPTION;
  -- salary variables
  n_salary emp.sal%TYPE;
  n_min_salary emp.sal%TYPE;
  n_max_salary emp.sal%TYPE;
  -- input employee id
  n_emp_id emp.empno%TYPE := &emp_id;

BEGIN
  n_min_salary:=2500;
  n_max_salary:=50000;
  SELECT sal INTO n_salary
  FROM emp
   WHERE empno = n_emp_id;
  IF n_salary < n_min_salary THEN
     RAISE BELOW_SALARY_RANGE;
  ELSIF n_salary > n_max_salary THEN
     RAISE ABOVE_SALARY_RANGE;
  END IF;
  dbms_output.put_line('Employee ' || n_emp_id ||
                       ' has salary $' || n_salary );
```

```
  EXCEPTION
    WHEN BELOW_SALARY_RANGE THEN
      dbms_output.put_line('Employee ' || n_emp_id ||
                                  ' has salary below the salary range');
    WHEN ABOVE_SALARY_RANGE THEN
      dbms_output.put_line('Employee ' || n_emp_id ||
                                  ' has salary above the salary range');
    WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee ' || n_emp_id || ' not found');
END;
/

OUTPUT:
Enter value for emp_id: 103
old  10:   n_emp_id emp.empno%TYPE := &emp_id;
new  10:   n_emp_id emp.empno%TYPE := 103;
Employee 103 has salary below the salary range

PL/SQL procedure successfully completed.

SQL> /
Enter value for emp_id: 104
old  10:   n_emp_id emp.empno%TYPE := &emp_id;
new  10:   n_emp_id emp.empno%TYPE := 104;
Employee 104 has salary $4200

PL/SQL procedure successfully completed.

SQL> /
Enter value for emp_id: 105
old  10:   n_emp_id emp.empno%TYPE := &emp_id;
new  10:   n_emp_id emp.empno%TYPE := 105;
Employee 105 has salary below the salary range

PL/SQL procedure successfully completed.

------------------------------The END----------------------------------------------------
```